

The Data Link Layer

Chapter 3

- Data Link Layer Design Issues
- Error Detection and Correction
- Elementary Data Link Protocols
- Sliding Window Protocols
- Example Data Link Protocols

Revised: August 2011

The Data Link Layer

Responsible for delivering frames of information over a single link

- Handles transmission errors and regulates the flow of data

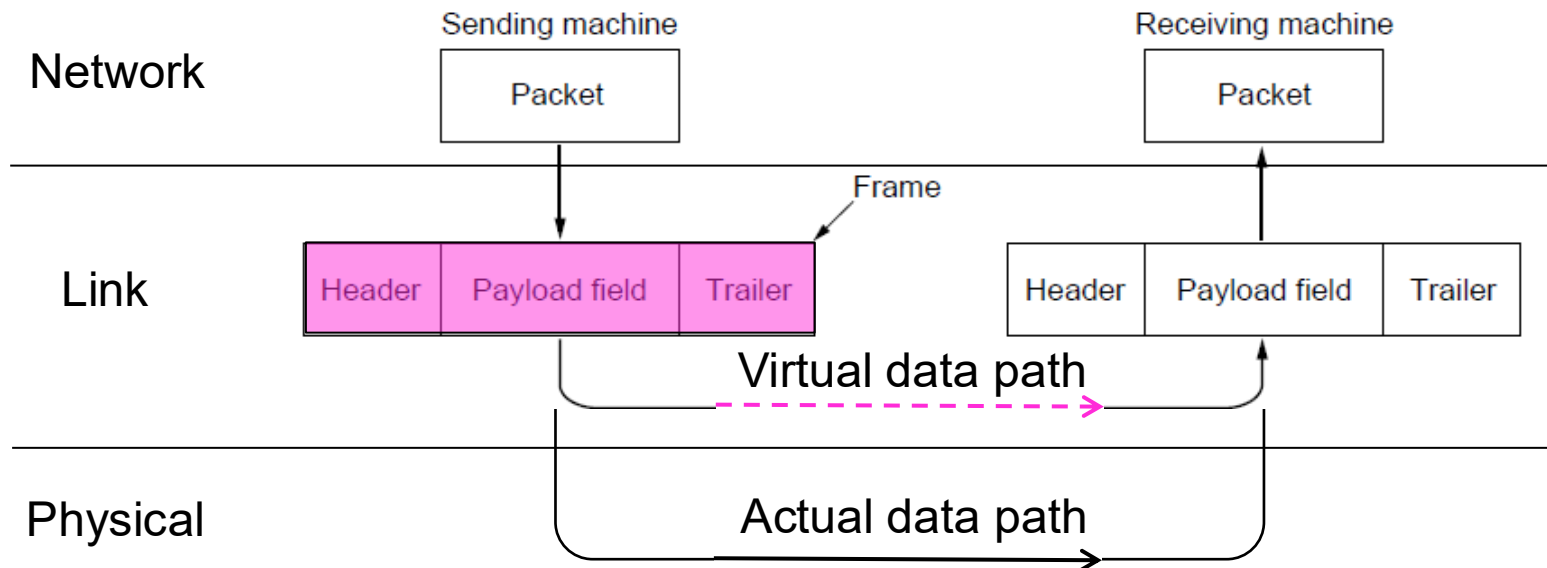
Application
Transport
Network
Link
Physical

Data Link Layer Design Issues

- Frames »
- Possible services »
- Framing methods »
- Error control »
- Flow control »

Frames

Link layer accepts packets from the network layer, and encapsulates them into frames that it sends using the physical layer; reception is the opposite process



Possible Services

Unacknowledged connectionless service

- Frame is sent with no connection / error recovery
- Ethernet is example

Acknowledged connectionless service

- Frame is sent with retransmissions if needed
- Example is 802.11

Acknowledged connection-oriented service

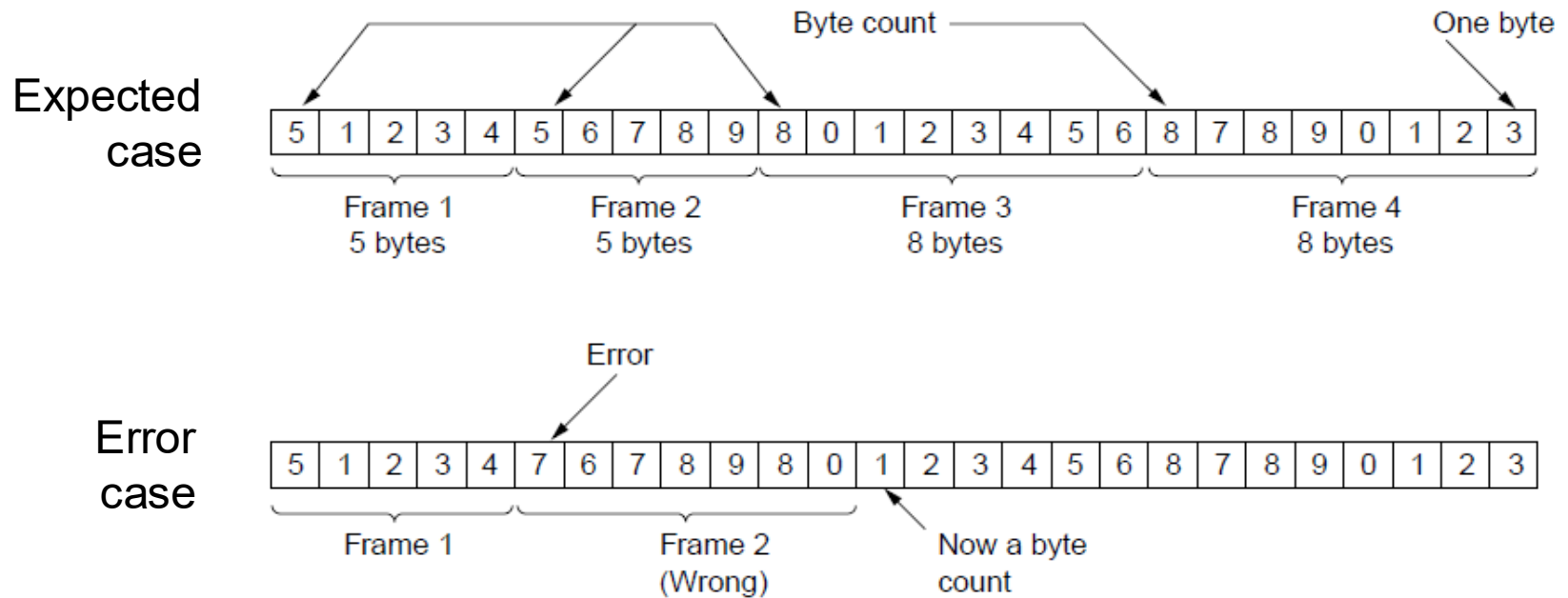
- Connection is set up; rare

Framing Methods

- Byte count »
- Flag bytes with byte stuffing »
- Flag bits with bit stuffing »
- Physical layer coding violations
 - Use non-data symbol to indicate frame

Framing – Byte count

- Frame begins with a count of the number of bytes in it
- Simple, but difficult to resynchronize after an error

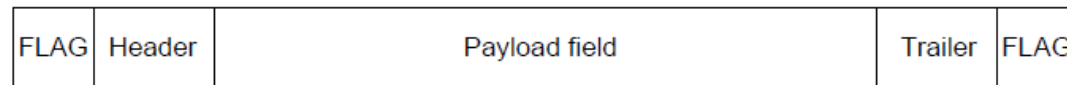


Framing – Byte stuffing

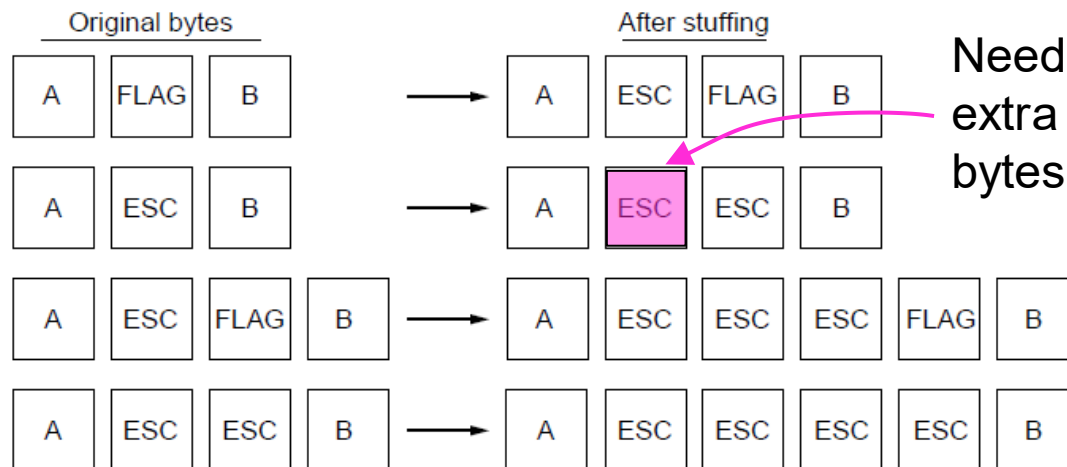
Special flag bytes delimit frames; occurrences of flags in the data must be stuffed (escaped)

- Longer, but easy to resynchronize after error

Frame
format



Stuffing
examples



Need to escape
extra ESCAPE
bytes too!

Framing – Bit stuffing

Stuffing done at the bit level:

- Frame flag has six consecutive 1s (not shown)
- On transmit, after five 1s in the data, a 0 is added
- On receive, a 0 after five 1s is deleted

Data bits 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

Transmitted bits with stuffing

0 1 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1 1 1 0 1 0 0 1 0

Stuffed bits

Error Control

Error control repairs frames that are received in error

- Requires errors to be detected at the receiver
- Typically retransmit the unacknowledged frames
- Timer protects against lost acknowledgements

Detecting errors and retransmissions are next topics.

Flow Control

Prevents a fast sender from out-pacing a slow receiver

- Receiver gives feedback on the data it can accept
- Rare in the Link layer as NICs run at “wire speed”
 - Receiver can take data as fast as it can be sent

Flow control is a topic in the Link and Transport layers.

Error Detection and Correction

Error codes add structured redundancy to data so errors can be either detected, or corrected.

Error correction codes:

- Hamming codes »
- Binary convolutional codes »
- Reed-Solomon and Low-Density Parity Check codes
 - Mathematically complex, widely used in real systems

Error detection codes:

- Parity »
- Checksums »
- Cyclic redundancy codes »

Error Bounds – Hamming distance

Code turns data of n bits into codewords of $n+k$ bits

Hamming distance is the minimum bit flips to turn one valid codeword into any other valid one.

- Example with 4 codewords of 10 bits ($n=2, k=8$):
 - 0000000000, 0000011111, 1111100000, and 1111111111
 - Hamming distance is 5

Bounds for a code with distance:

- $2d+1$ – can correct d errors (e.g., 2 errors above)
- $d+1$ – can detect d errors (e.g., 4 errors above)

10011010

(1, 2, 4, 8, 16)

<u>1</u>	<u>0</u>	<u>0</u>	<u>1</u>	<u>1</u>	<u>0</u>	<u>1</u>	<u>0</u>
P1	P2	d3	P4	d5	d6	d7	P8

P1 $\Rightarrow$? 10111 $\Rightarrow$ 0

P2 $\Rightarrow$? 10101 $\Rightarrow$ 1

P4 $\Rightarrow$? 0010 $\Rightarrow$ 1

P8 $\Rightarrow$? 1010 $\Rightarrow$ 0

0	1	1	1	0	0	1	0	1	1	1	0
P1	P2	d3	P4	d5	d6	d7	P8	d9	d10	d11	d12

↓ error

P1 check, take 1 bit skip 1 bit

P1 $\Rightarrow$? 10111 $\Rightarrow$ 0 ✓

P2 $\Rightarrow$? 10111 $\Rightarrow$ 0 X

P4 $\Rightarrow$? 0010 $\Rightarrow$ 1 ✓

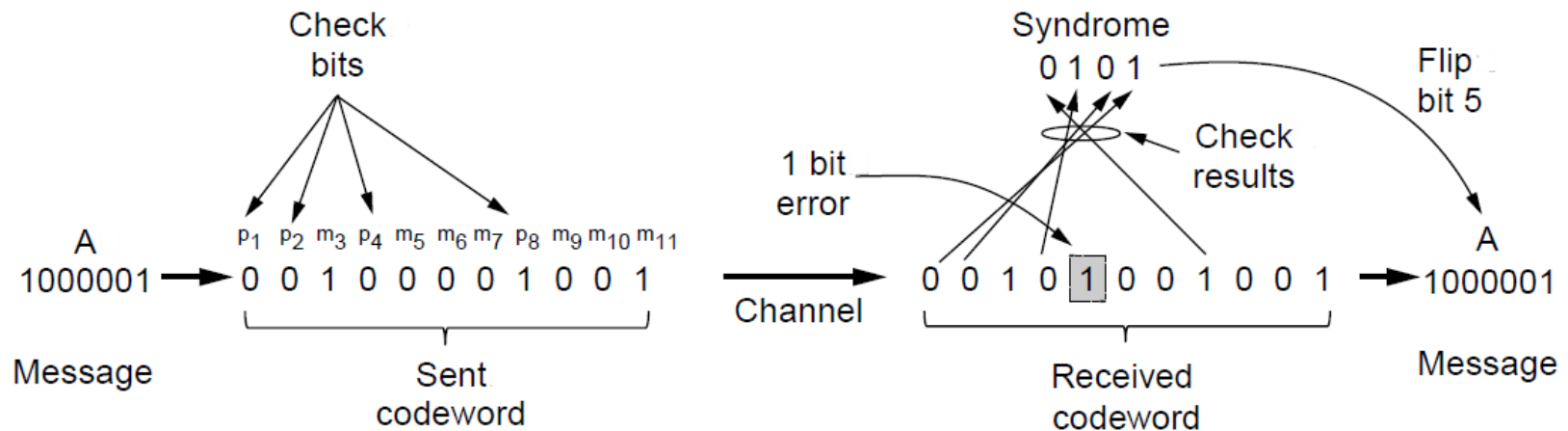
P8 $\Rightarrow$? 1110 $\Rightarrow$ 1 X

P2 + P8 not correct 2 + 8 = 10th bit is dead

Error Correction – Hamming code

Hamming code gives a simple way to add check bits and correct up to a single bit error:

- Check bits are parity over subsets of the codeword
- Recomputing the parity sums (syndrome) gives the position of the error to flip, or 0 if there is no error

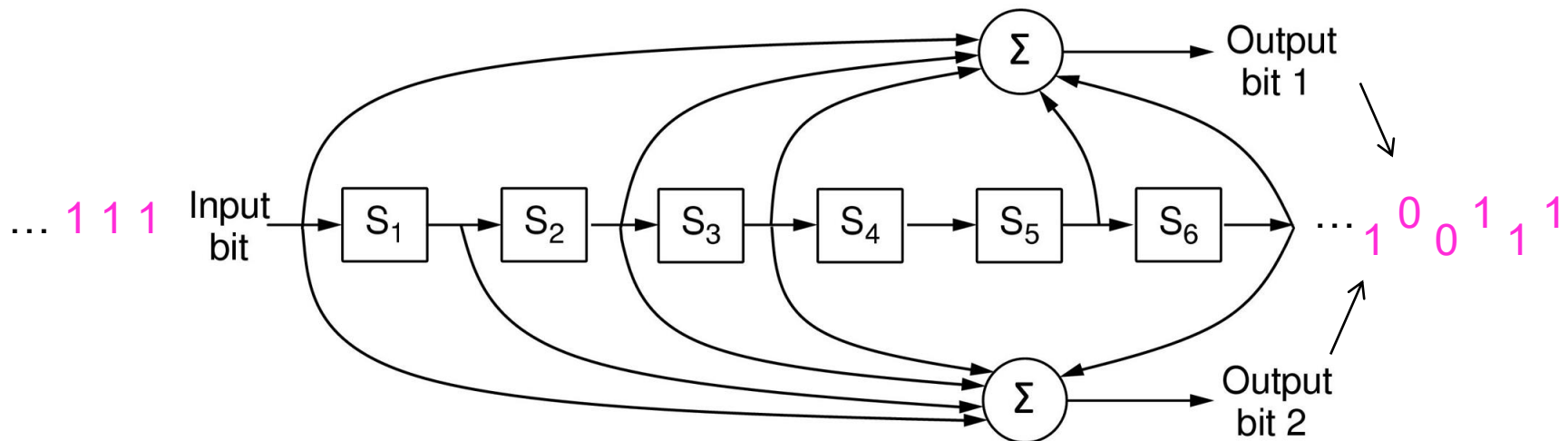


(11, 7) Hamming code adds 4 check bits and can correct 1 error

Error Correction – Convolutional codes

Operates on a stream of bits, keeping internal state

- Output stream is a function of all preceding input bits
- Bits are decoded with the Viterbi algorithm



Popular NASA binary convolutional code (rate = $\frac{1}{2}$) used in 802.11

Error Detection – Parity (1)

Parity bit is added as the modulo 2 sum of data bits

- Equivalent to XOR; this is even parity
- Ex: 1110000 → 11100001
- Detection checks if the sum is wrong (an error)

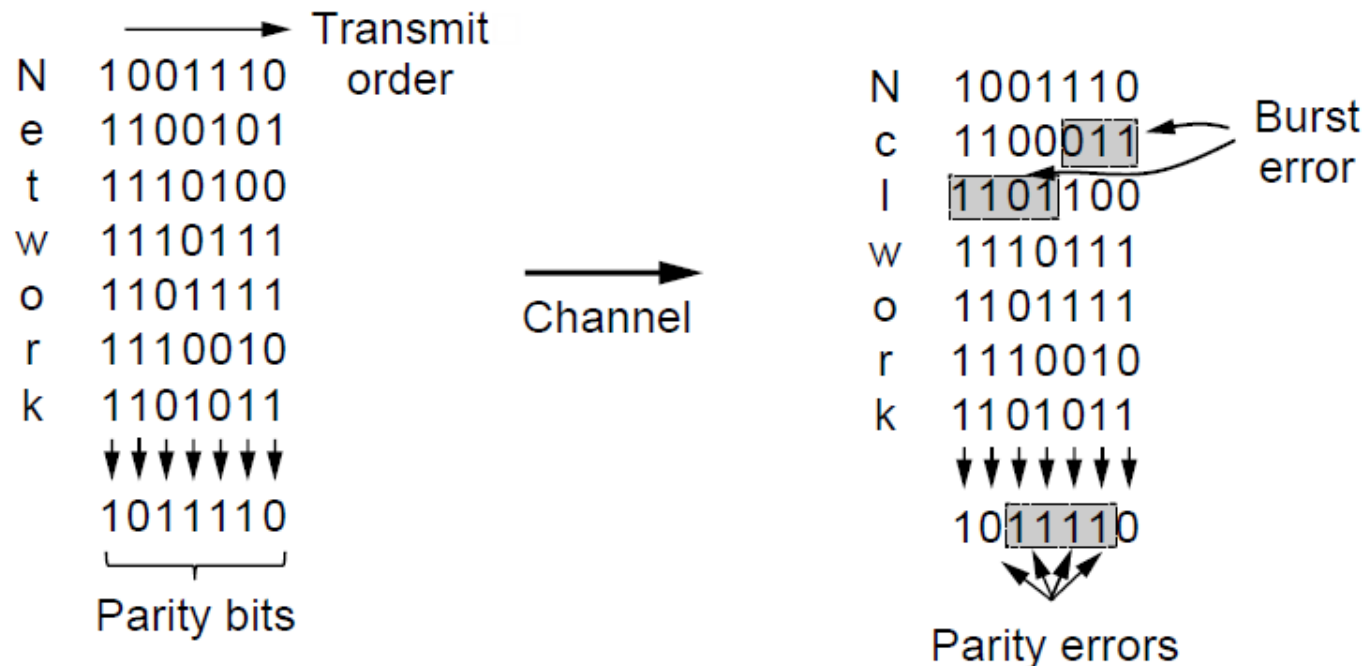
Simple way to detect an *odd* number of errors

- Ex: 1 error, 11100101; detected, sum is wrong
- Ex: 3 errors, 11011001; detected sum is wrong
- Ex: 2 errors, 11101101; *not detected*, sum is right!
- Error can also be in the parity bit itself
- Random errors are detected with probability $\frac{1}{2}$

Error Detection – Parity (2)

Interleaving of N parity bits detects burst errors up to N

- Each parity sum is made over non-adjacent bits
- An even burst of up to N errors will not cause it to fail



Error Detection – Checksums

Checksum treats data as N-bit words and adds N check bits that are the modulo 2^N sum of the words

- Ex: Internet 16-bit 1s complement checksum

Properties:

- Improved error detection over parity bits
- Detects bursts up to N errors
- Detects random errors with probability $1-2^{-N}$
- Vulnerable to systematic errors, e.g., added zeros

Original Data

10011001	11100010	00100100	10000100
----------	----------	----------	----------

1
k=4, m=8

Sender

```

1  10011001
2  11100010
   -----
   101111011
     1
   -----
    01111100
3  00100100
   -----
    10100000
4  10000100
   -----
   100100100
     1
   -----
Sum: 00100101

```

Checksum: 11011010

Receiver

```

1  10011001
2  11100010
   -----
   101111011
     1
   -----
    01111100
3  00100100
   -----
    10100000
4  10000100
   -----
   100100100
     1
   -----
    00100101
    11011010
   -----
Sum: 11111111

```

Complement: 00000000

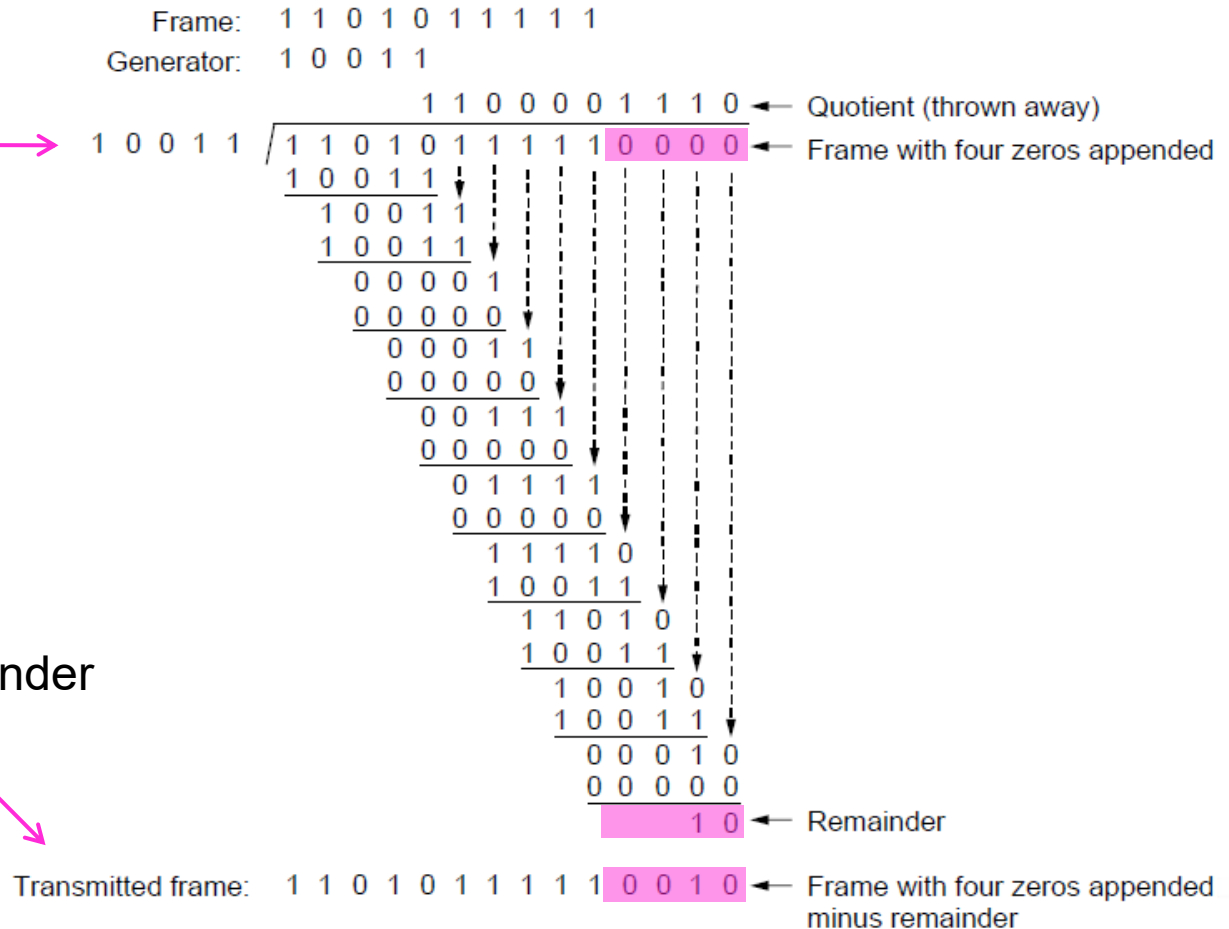
Conclusion: Accept Data

Error Detection – CRCs (1)

Adds bits so that transmitted frame viewed as a polynomial is evenly divisible by a generator polynomial

Start by adding
0s to frame
and try dividing

Offset by any remainder
to make it evenly
divisible



Error Detection – CRCs (2)

Based on standard polynomials:

- Ex: Ethernet 32-bit CRC is defined by:

$$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x^1 + 1$$

- Computed with simple shift/XOR circuits

Stronger detection than checksums:

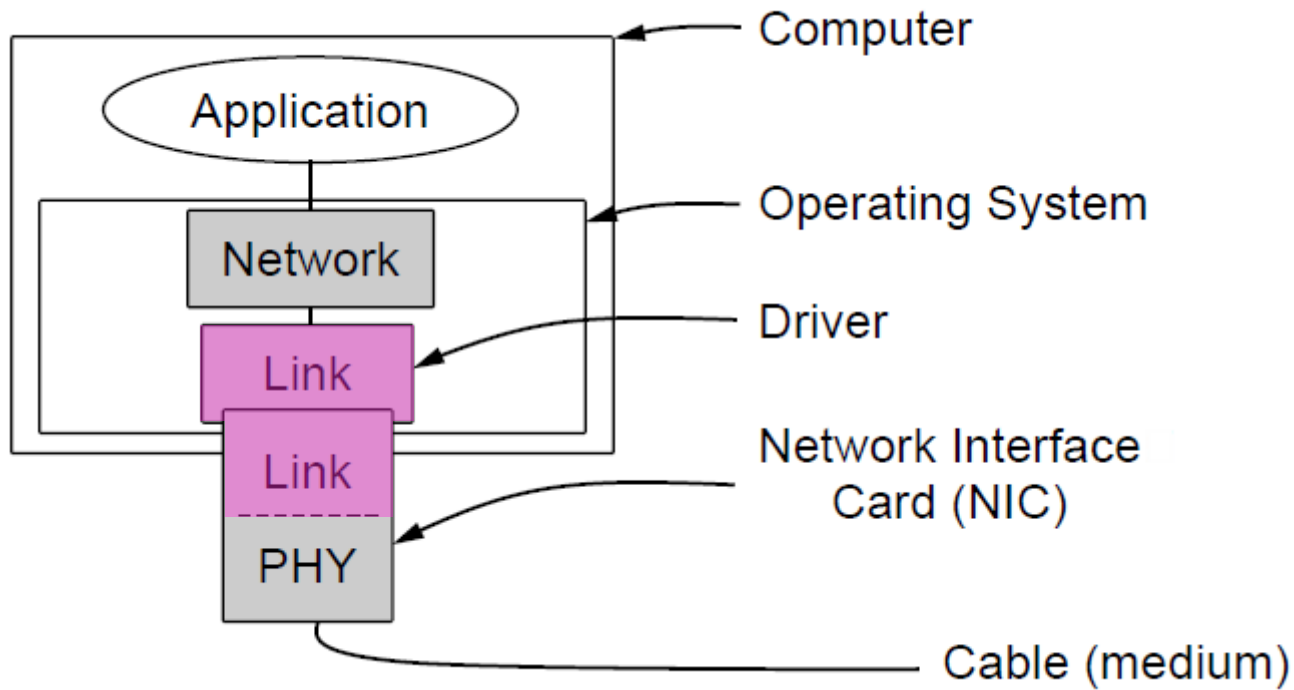
- E.g., can detect all double bit errors
- Not vulnerable to systematic errors

Elementary Data Link Protocols

- Link layer environment »
- Utopian Simplex Protocol »
- Stop-and-Wait Protocol for Error-free channel »
- Stop-and-Wait Protocol for Noisy channel »

Link layer environment (1)

Commonly implemented as NICs and OS drivers;
network layer (IP) is often OS software



Link layer environment (2)

Link layer protocol implementations use library functions

- See code (`protocol.h`) for more details

Group	Library Function	Description
Network layer	<code>from_network_layer(&packet)</code> <code>to_network_layer(&packet)</code> <code>enable_network_layer()</code> <code>disable_network_layer()</code>	Take a packet from network layer to send Deliver a received packet to network layer Let network cause “ready” events Prevent network “ready” events
Physical layer	<code>from_physical_layer(&frame)</code> <code>to_physical_layer(&frame)</code>	Get an incoming frame from physical layer Pass an outgoing frame to physical layer
Events & timers	<code>wait_for_event(&event)</code> <code>start_timer(seq_nr)</code> <code>stop_timer(seq_nr)</code> <code>start_ack_timer()</code> <code>stop_ack_timer()</code>	Wait for a packet / frame / timer event Start a countdown timer running Stop a countdown timer from running Start the ACK countdown timer Stop the ACK countdown timer

Utopian Simplex Protocol

An optimistic protocol (p1) to get us started

- Assumes no errors, and receiver as fast as sender
- Considers one-way data transfer

```
void sender1(void)
{
    frame s;
    packet buffer;

    while (true) {
        from_network_layer(&buffer);
        s.info = buffer;
        to_physical_layer(&s);
    }
}
```

Sender loops blasting frames

```
void receiver1(void)
{
    frame r;
    event_type event;

    while (true) {
        wait_for_event(&event);
        from_physical_layer(&r);
        to_network_layer(&r.info);
    }
}
```

Receiver loops eating frames

- That's it, no error or flow control ...

Stop-and-Wait – Error-free channel

Protocol (p2) ensures sender can't outpace receiver:

- Receiver returns a dummy frame (ack) when ready
- Only one frame out at a time – called stop-and-wait
- We added flow control!

```
void sender2(void)
{
    frame s;
    packet buffer;
    event_type event;

    while (true) {
        from_network_layer(&buffer);
        s.info = buffer;
        to_physical_layer(&s);
        wait_for_event(&event);
    }
}
```

Sender waits to for ack after
passing frame to physical layer

```
void receiver2(void)
{
    frame r, s;
    event_type event;
    while (true) {
        wait_for_event(&event);
        from_physical_layer(&r);
        to_network_layer(&r.info);
        to_physical_layer(&s);
    }
}
```

Receiver sends ack after passing
frame to network layer

Stop-and-Wait – Noisy channel (1)

ARQ (Automatic Repeat reQuest) adds error control

- Receiver acks frames that are correctly delivered
- Sender sets timer and resends frame if no ack)

For correctness, frames and acks must be numbered

- Else receiver can't tell retransmission (due to lost ack or early timer) from new frame
- For stop-and-wait, 2 numbers (1 bit) are sufficient

Stop-and-Wait – Noisy channel (2)

Sender loop (p3):

Send frame (or retransmission)
Set timer for retransmission
Wait for ack or timeout

If a good ack then set up for the
next frame to send (else the old
frame will be retransmitted)

```
void sender3(void) {  
    seq_nr next_frame_to_send;  
    frame s;  
    packet buffer;  
    event_type event;  
  
    next_frame_to_send = 0;  
    from_network_layer(&buffer);  
    while (true) {  
        s.info = buffer;  
        s.seq = next_frame_to_send;  
        to_physical_layer(&s);  
        start_timer(s.seq);  
        wait_for_event(&event);  
        if (event == frame_arrival) {  
            from_physical_layer(&s);  
            if (s.ack == next_frame_to_send) {  
                stop_timer(s.ack);  
                from_network_layer(&buffer);  
                inc(next_frame_to_send);  
            }  
        }  
    }  
}
```

Stop-and-Wait – Noisy channel (3)

Receiver loop (p3):

```
void receiver3(void)
{
    seq_nr frame_expected;
    frame r, s;
    event_type event;

    frame_expected = 0;
    while (true) {
        wait_for_event(&event);
        if (event == frame_arrival) {
            from_physical_layer(&r);
            if (r.seq == frame_expected) {
                to_network_layer(&r.info);
                inc(frame_expected);
            }
            s.ack = 1 - frame_expected;
            to_physical_layer(&s);
        }
    }
}
```

Wait for a frame →

If it's new then take it and advance expected frame {

Ack current frame →

Sliding Window Protocols

- Sliding Window concept »
- One-bit Sliding Window »
- Go-Back-N »
- Selective Repeat »

Sliding Window concept (1)

Sender maintains window of frames it can send

- Needs to buffer them for possible retransmission
- Window advances with next acknowledgements

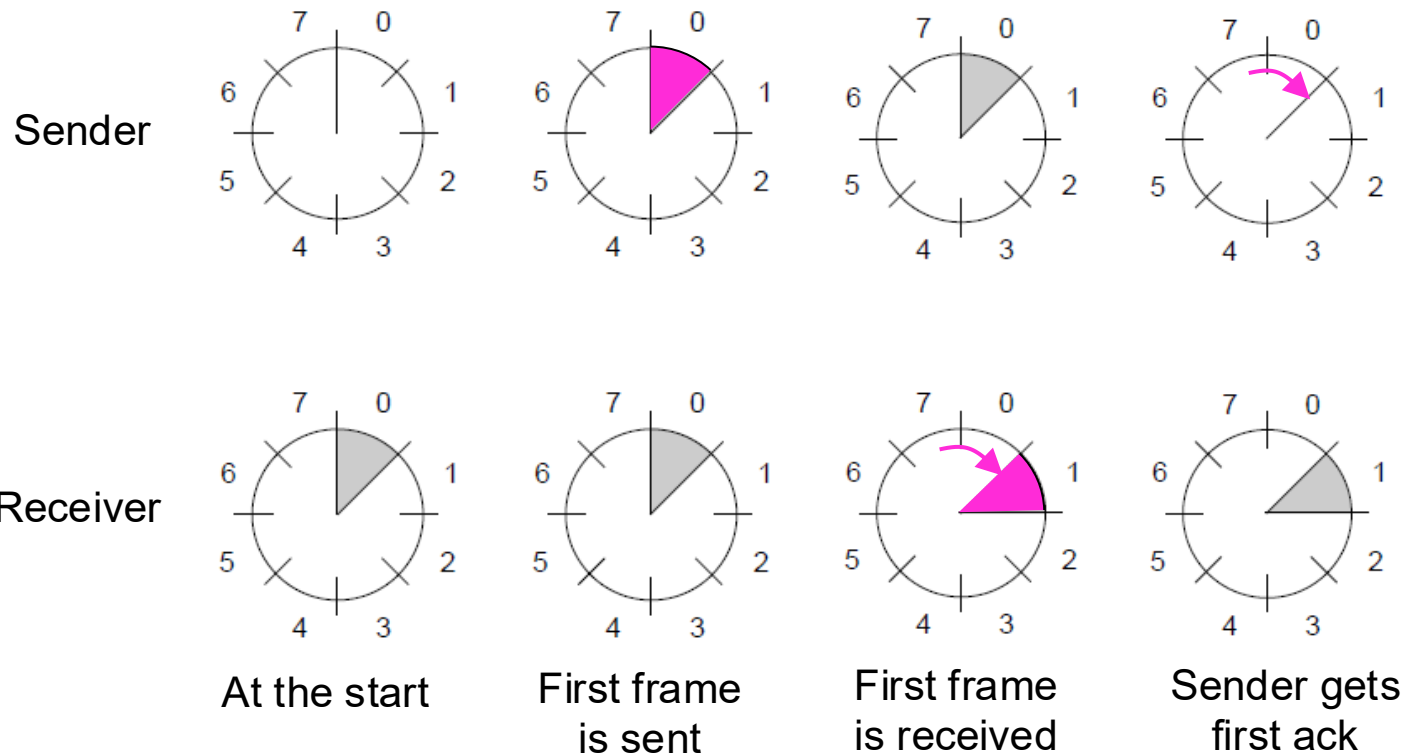
Receiver maintains window of frames it can receive

- Needs to keep buffer space for arrivals
- Window advances with in-order arrivals

Sliding Window concept (2)

A sliding window advancing at the sender and receiver

- Ex: window size is 1, with a 3-bit sequence number.



Sliding Window concept (3)

Larger windows enable pipelining for efficient link use

- Stop-and-wait ($w=1$) is inefficient for long links
- Best window (w) depends on bandwidth-delay (BD)
- Want $w \geq 2BD+1$ to ensure high link utilization

Pipelining leads to different choices for errors/buffering

- We will consider Go-Back-N and Selective Repeat

One-Bit Sliding Window (1)

Transfers data in both directions with stop-and-wait

- Piggybacks acks on reverse data frames for efficiency
- Handles transmission errors, flow control, early timers

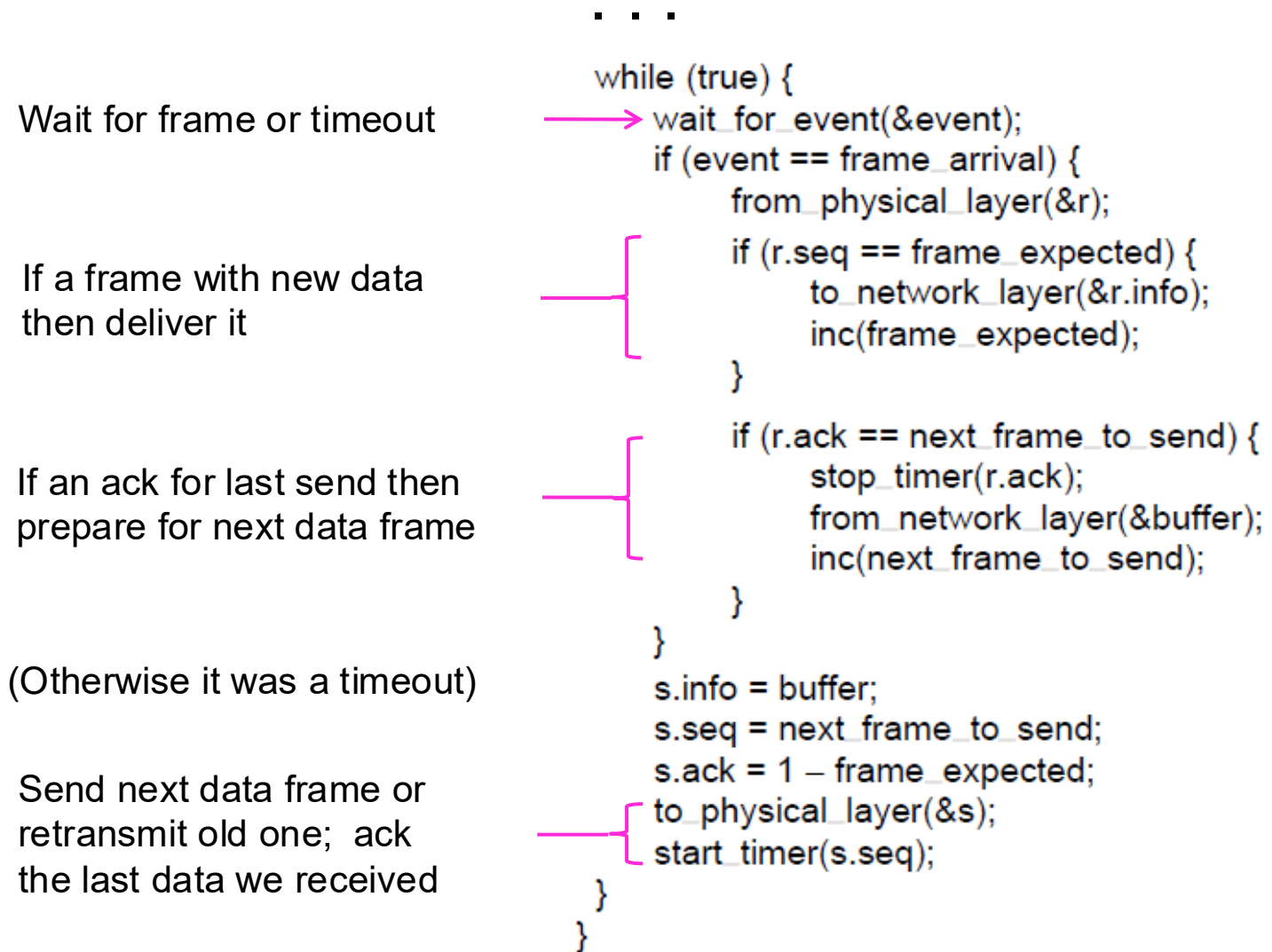
Each node is sender
and receiver (p4):

```
void protocol4 (void) {  
    seq_nr next_frame_to_send;  
    seq_nr frame_expected;  
    frame r, s;  
    packet buffer;  
    event_type event;  
  
    next_frame_to_send = 0;  
    frame_expected = 0;  
    from_network_layer(&buffer);  
    s.info = buffer;  
    s.seq = next_frame_to_send;  
    s.ack = 1 - frame_expected;  
    to_physical_layer(&s);  
    start_timer(s.seq);  
  
    .  
    .  
    .  
}
```

Prepare first frame — {

Launch it, and set timer — {

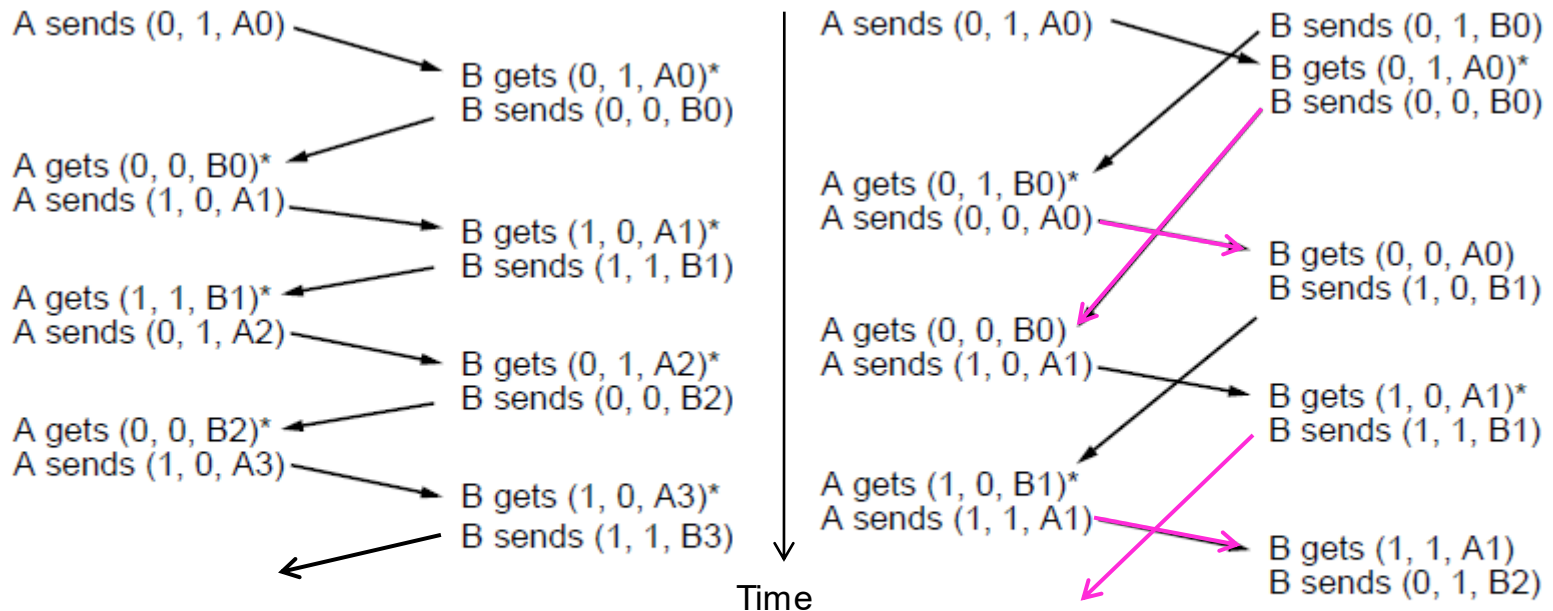
One-Bit Sliding Window (2)



One-Bit Sliding Window (3)

Two scenarios show subtle interactions exist in p4:

- Simultaneous start [right] causes correct but slow operation compared to normal [left] due to duplicate transmissions.



Notation is (seq, ack, frame number). Asterisk indicates frame accepted by network layer .

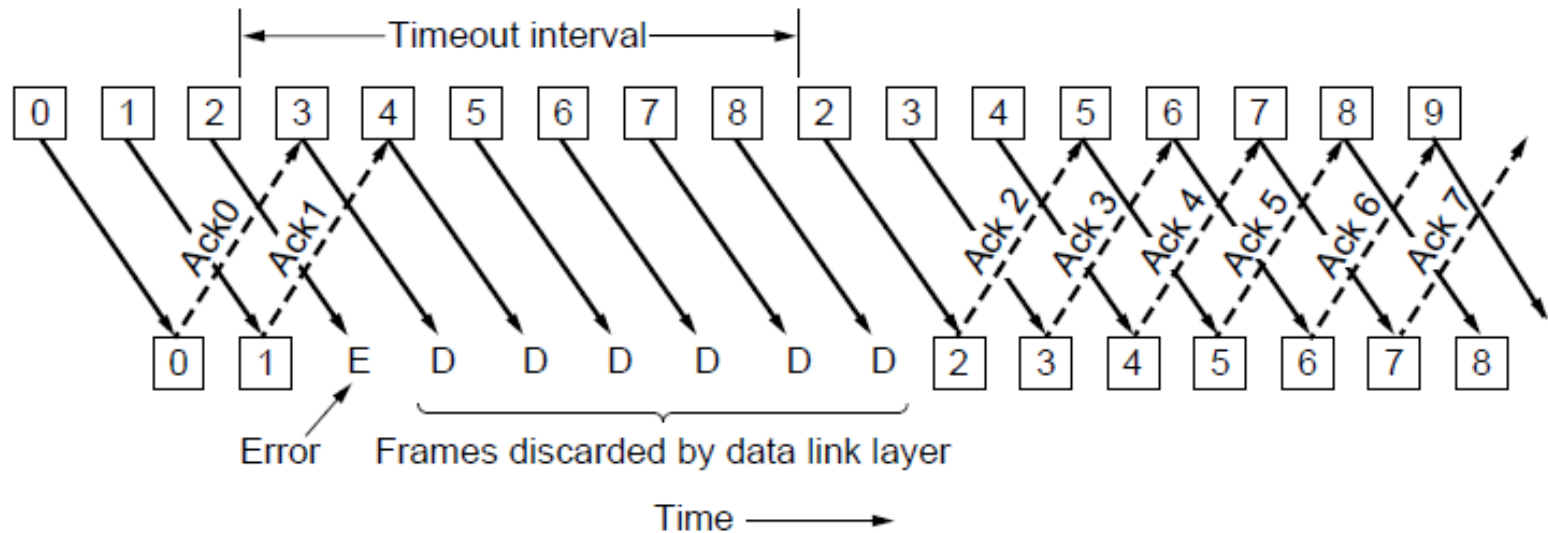
Normal case

Correct, but poor performance

Go-Back-N (1)

Receiver only accepts/acks frames that arrive in order:

- Discards frames that follow a missing/errored frame
- Sender times out and resends all outstanding frames



Go-Back-N (2)

Tradeoff made for Go-Back-N:

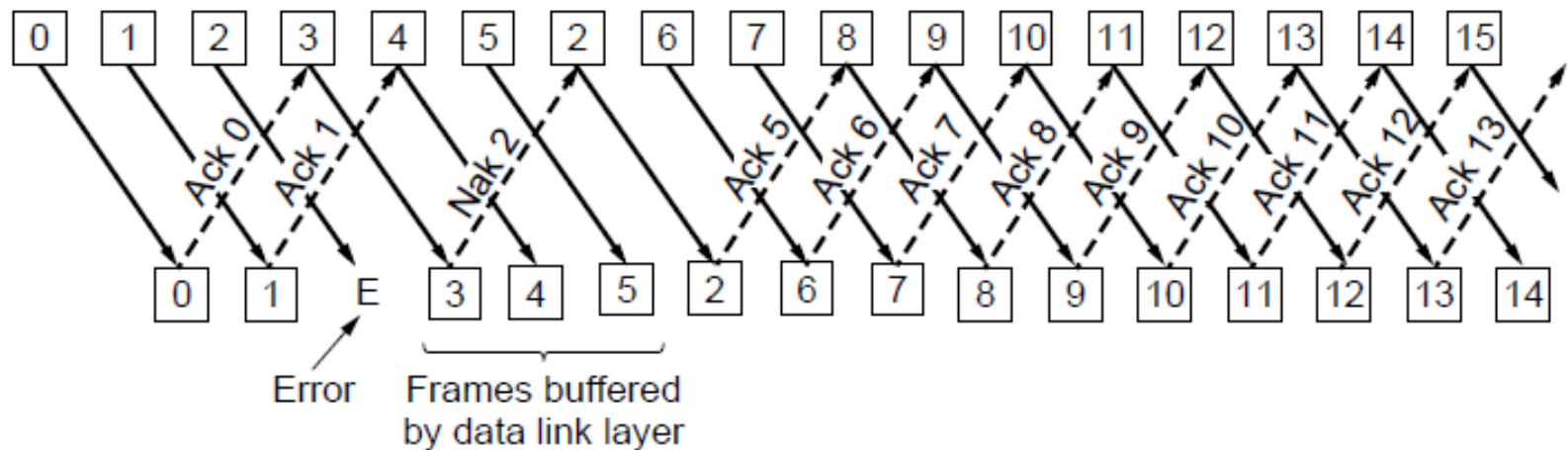
- Simple strategy for receiver; needs only 1 frame
- Wastes link bandwidth for errors with large windows; entire window is retransmitted

Implemented as p5 (see code in book)

Selective Repeat (1)

Receiver accepts frames anywhere in receive window

- Cumulative ack indicates highest in-order frame
- NAK (negative ack) causes sender retransmission of a missing frame before a timeout resends window



Selective Repeat (2)

Tradeoff made for Selective Repeat:

- More complex than Go-Back-N due to buffering at receiver and multiple timers at sender
- More efficient use of link bandwidth as only lost frames are resent (with low error rates)

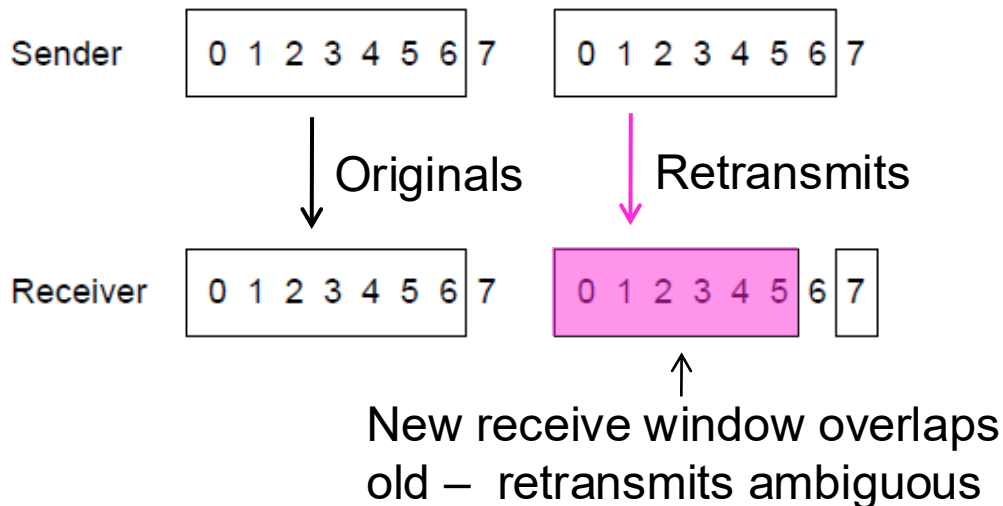
Implemented as p6 (see code in book)

Selective Repeat (3)

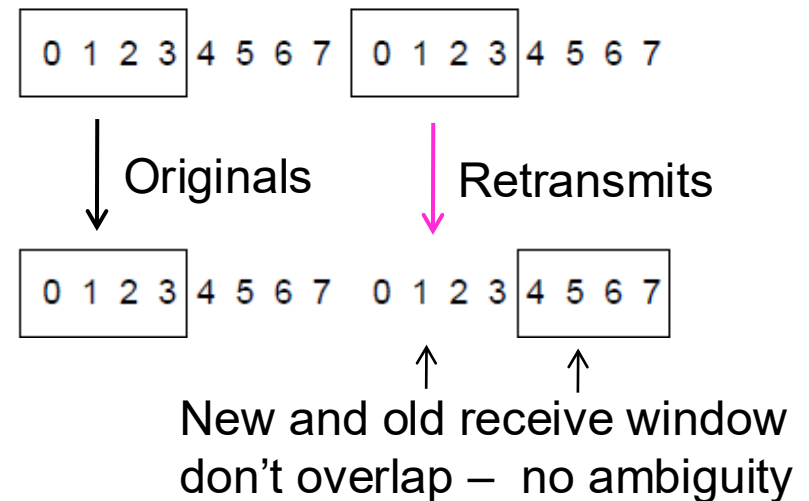
For correctness, we require:

- Sequence numbers (s) at least twice the window (w)

Error case ($s=8, w=7$) – too few sequence numbers



Correct ($s=8, w=4$) – enough sequence numbers



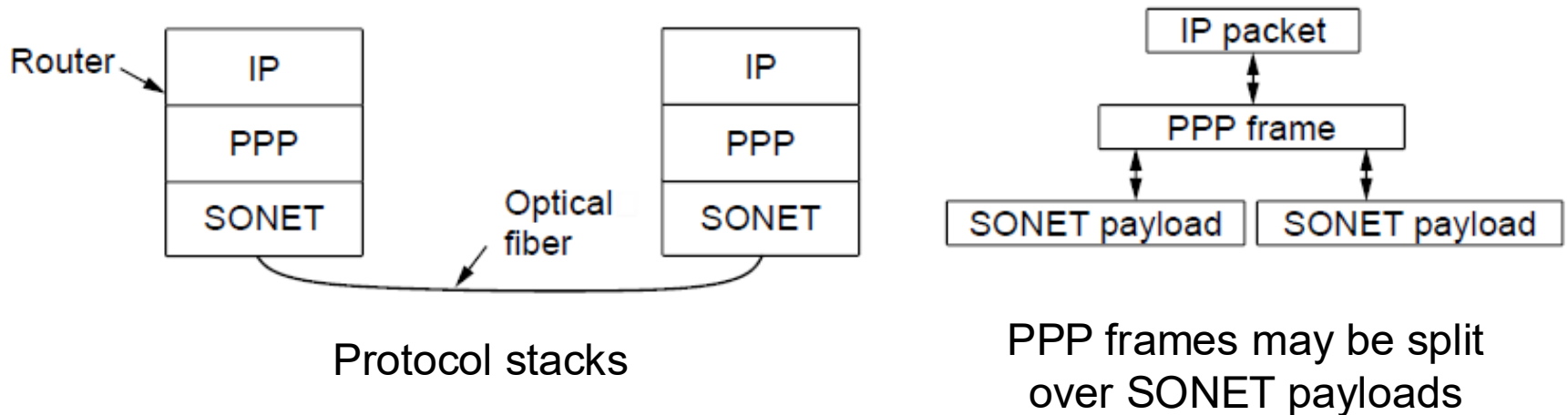
Example Data Link Protocols

- Packet over SONET »
- PPP (Point-to-Point Protocol) »
- ADSL (Asymmetric Digital Subscriber Loop) »

Packet over SONET

Packet over SONET is the method used to carry IP packets over SONET optical fiber links

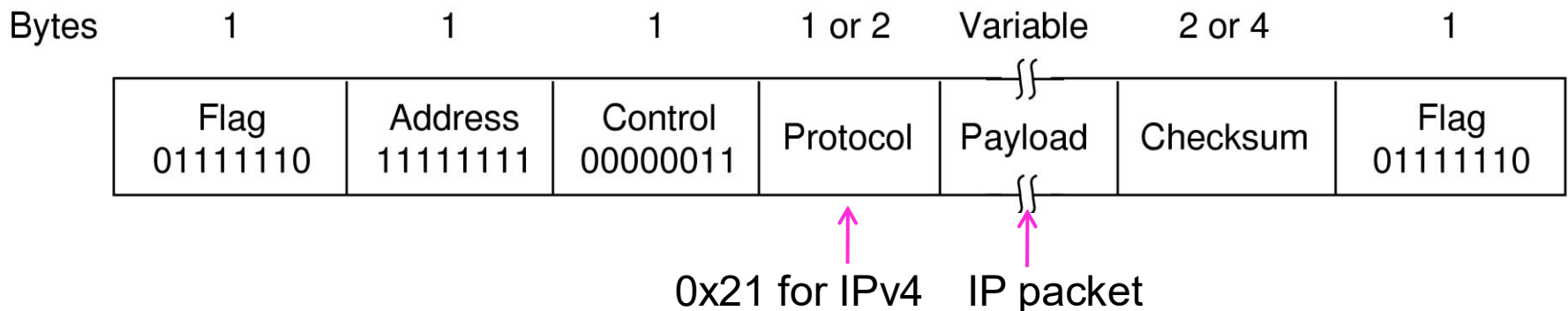
- Uses PPP (Point-to-Point Protocol) for framing



PPP (1)

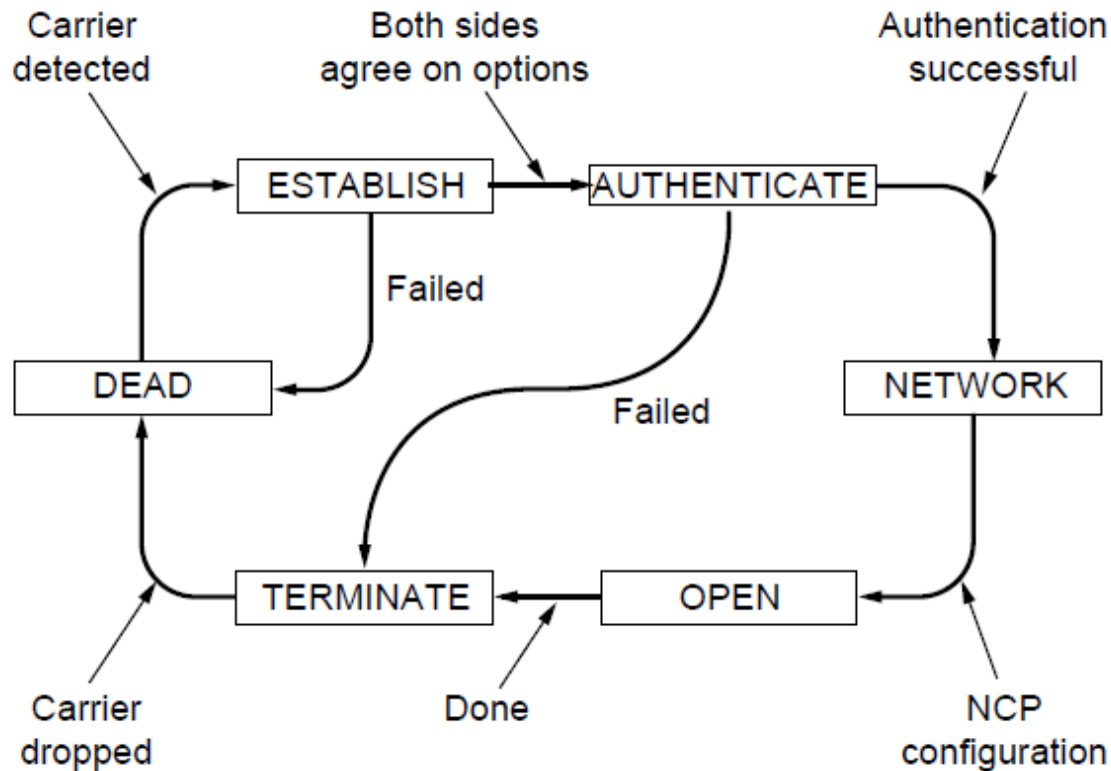
PPP (Point-to-Point Protocol) is a general method for delivering packets across links

- Framing uses a flag (0x7E) and byte stuffing
- “Unnumbered mode” (connectionless unacknowledged service) is used to carry IP packets
- Errors are detected with a checksum



PPP (2)

A link control protocol brings the PPP link up/down

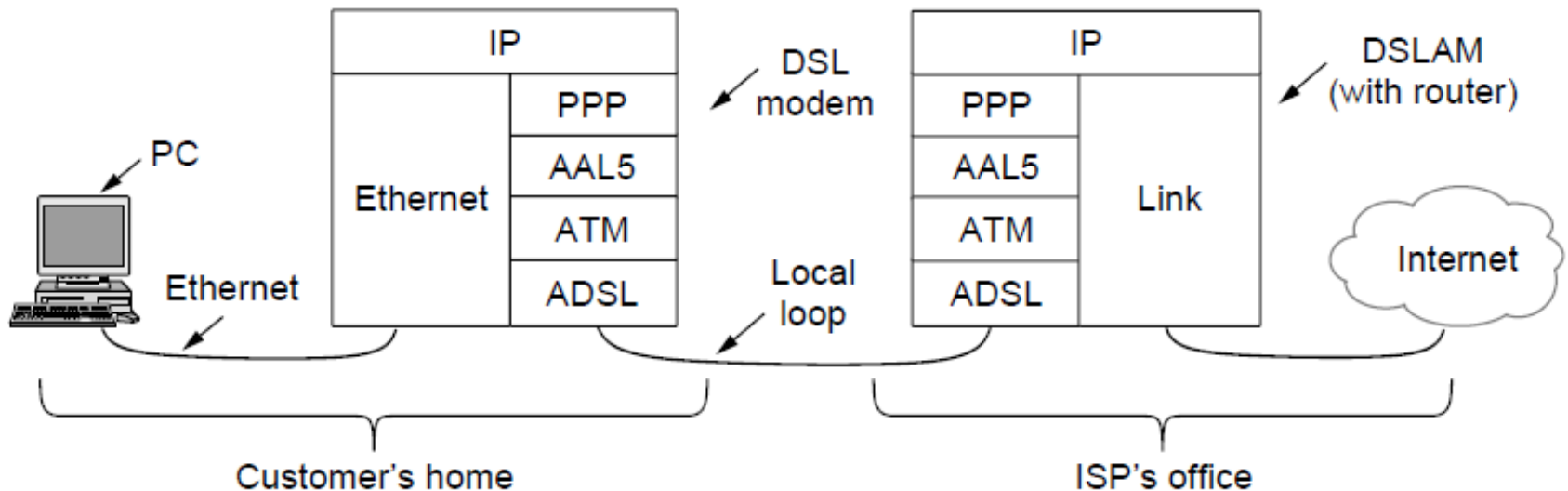


State machine for link control

ADSL (1)

Widely used for broadband Internet over local loops

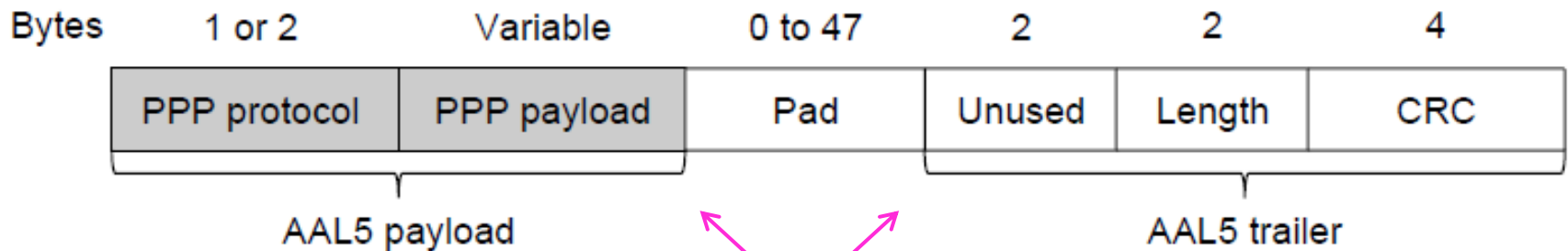
- ADSL runs from modem (customer) to DSLAM (ISP)
- IP packets are sent over PPP and AAL5/ATM (over)



ADSL (2)

PPP data is sent in AAL5 frames over ATM cells:

- ATM is a link layer that uses short, fixed-size cells (53 bytes); each cell has a virtual circuit identifier
- AAL5 is a format to send packets over ATM
- PPP frame is converted to a AAL5 frame (PPPoA)



AAL5 frame is divided into 48 byte pieces, each of which goes into one ATM cell with 5 header bytes

End

Chapter 3